Universidade Federal de São Paulo (UNIFESP)

Cálculo Numérico

Prof. Luiz Leduino de Salles Neto

Trabalho Computacional 3: interpolação polinomial e ajuste de curvas por mínimos quadrados

Relatório acadêmico

Integrantes

Davi Giuberti de Souza	RA 168631
Gabriel Lemos da Silva	RA 140971
Mayara Martinez Boaro	RA 188197
Rodrigo Ezequiel Silva dos Anjos	RA 188215

São José dos Campos

2026

Sumário

1 Introdução **2**

2 Objetivos **2**

2.1 Objetivo geral 2

2.2 Objetivos específicos 2

3 Fundamentação teórica **3**

3.1 Descrição do método 3

3.2 Algoritmo 5

4 Implementação computacional **6**

4.1 Trecho de código 6

5 Experimentos numéricos **7**

5.1 Configuração dos testes 7

5.2 Resultados 8

5.3 Gráficos e figuras 13

6 Análise e discussão **15**

6.1 Parte 1: interpolação 15

6.2 Parte 2: mínimos quadrados 16

7 Conclusões **17**

8 Referências **18**

A Código-fonte e instruções de execução **18**

B Dados e problemas de teste **27**

C Material complementar **28**

1 Introdução

O TC3 tem duas partes. A parte 1 (exercícios 1 a 4) é de interpolação polinomial e a parte 2 (exercícios 2 a 6, mantendo a numeração do enunciado) é de ajuste de curvas por mínimos quadrados. Todos os exercícios vêm do livro de Kharab e Guenther (2018).

Nas duas partes queremos uma curva que represente uma tabela de pontos, mas o critério muda. A interpolação procura "um polinômio simples que passe exatamente pelos pontos" (SALLES NETO, 2026b, slide 3), e serve para estimar valores entre os dados de uma tabela. A tabela de pressão de vapor da água do exercício 3 é um caso desses, e o slide 4 cita tabelas de vapor d'água entre as aplicações. No ajuste por mínimos quadrados a curva passa entre os pontos, com os coeficientes que deixam mínima a soma dos quadrados dos erros (SALLES NETO, 2026a). Ele é mais adequado quando os dados têm erro de medição ou quando se quer um polinômio de grau baixo para muitos pontos.

Para interpolar usamos a forma de Newton com diferenças divididas. A tabela custa $O(n^2)$ operações e dispensa o sistema de Vandermonde, que é mal condicionado (slide 38). Nos mínimos quadrados montamos as equações normais e resolvemos o sistema linear resultante por eliminação de Gauss, como na aula de sistemas lineares.

A teoria está resumida na Seção 3 e o código na Seção 4. Os resultados de cada exercício aparecem na Seção 5, e a Seção 6 discute os casos em que o método funcionou bem e o exercício 4, em que não funcionou. O código completo e os dados ficaram nos apêndices.

2 Objetivos

2.1 Objetivo geral

Resolver os exercícios do TC3 implementando em Python a interpolação na forma de Newton e o ajuste por mínimos quadrados, e verificar em quais exercícios cada método dá bons resultados e em quais não dá.

2.2 Objetivos específicos

- Implementar a tabela de diferenças divididas e a avaliação do polinômio na forma de Newton.
- Calcular os polinômios interpoladores da parte 1 e comparar as estimativas com os valores exatos ou conhecidos dados no enunciado.
- Montar e resolver as equações normais da parte 2 e calcular o erro E de cada ajuste.
- Conferir os resultados com o sistema de Vandermonde, com a função `interpola` do laboratório e com o `numpy.polyfit`.

- Analisar como o grau do polinômio e a distribuição dos nós afetam o erro.

3 Fundamentação teórica

As fórmulas abaixo foram tiradas dos slides de interpolação (SALLES NETO, 2026b) e de ajuste de curvas (SALLES NETO, 2026a). O número do slide de onde cada uma saiu está entre parênteses.

3.1 Descrição do método

3.1.1 Interpolação polinomial

Dados $n+1$ pontos com abscissas distintas (x_0, y_0), (x_1, y_1), …, (x_n, y_n), o problema de interpolação (slide 6) é encontrar um polinômio p de grau $\leq n$ tal que

$$p(x_i) = y_i, \qquad i = 0, 1, \ldots, n.$$

Os x_i são os nós e, em geral, $y_i = f(x_i)$. Se os nós são distintos, existe um único $p \in \mathcal{P}_n$ que satisfaz essas condições (slide 8). A unicidade sai de um argumento curto. Se p e q interpolam os mesmos dados, $r = p - q$ tem grau $\leq n$ e se anula nos $n+1$ nós, e um polinômio de grau $\leq n$ com $n+1$ raízes só pode ser $r \equiv 0$. Usamos esse teorema várias vezes na discussão.

Escrevendo $p(x) = a_0 + a_1 x + \cdots + a_n x^n$, as condições $p(x_i) = y_i$ viram o sistema $Va = y$, com a matriz de Vandermonde V (slide 9):

$$\begin{bmatrix} 1 & x_0 & \cdots & x_0^n \\ \vdots & \vdots & & \vdots \\ 1 & x_n & \cdots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ \vdots \\ y_n \end{bmatrix}, \qquad \det V = \prod_{i<j} (x_j - x_i) \neq 0.$$

O slide 38 avisa que essa matriz é mal condicionada, e isso piora quando os x_i são grandes ou quando n cresce.

A forma de Newton (slide 16) não passa por esse sistema. O polinômio é escrito como

$$p(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_n(x - x_0)\cdots(x - x_{n-1}).$$

e os coeficientes são as diferenças divididas (slide 17):

$$f[x_i] = y_i, \qquad f[x_i, \ldots, x_j] = \frac{f[x_{i+1}, \ldots, x_j] - f[x_i, \ldots, x_{j-1}]}{x_j - x_i}, \qquad c_k = f[x_0, x_1, \ldots, x_k].$$

Na prática as diferenças são calculadas numa tabela, uma coluna de cada vez. Pela regra

do slide 19, o elemento da linha de x_i na coluna de ordem k é

$$\frac{\text{(vizinha à esquerda)} - \text{(a de cima dela)}}{x_i - x_{i-k}},$$

e os coeficientes c_k ficam na diagonal da tabela (slide 24). Se entrar um ponto novo, os coeficientes antigos continuam valendo e aparece só mais um termo (slide 26).

O erro da interpolação é dado nos slides 28 e 29. Se $f \in C^{n+1}[a,b]$ e os nós estão em $[a,b]$, para cada $x \in [a,b]$ existe $\xi \in (a,b)$ tal que

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}\prod_{i=0}^{n}(x - x_i) \qquad \text{e} \qquad |f(x) - p_n(x)| \leq \frac{M_{n+1}}{(n+1)!}\left|\prod_{i=0}^{n}(x - x_i)\right|,$$

com $M_{n+1} = \max_{[a,b]} |f^{(n+1)}|$. A cota depende de quanto f "entorta" (M_{n+1}), do fatorial $(n+1)!$ e do produto $\prod(x - x_i)$, e esse último depende só dos nós. Por isso aumentar o número de pontos nem sempre reduz o erro: M_{n+1} pode crescer muito, e o produto fica grande perto das pontas do intervalo. É o fenômeno de Runge (slides 32 a 35). Fora do intervalo dos dados o produto cresce ainda mais rápido, e extrapolar com grau alto é perigoso (slide 37).

3.1.2 Ajuste de curvas por mínimos quadrados

Quando os dados têm erro, não faz sentido exigir que a curva passe por todos eles. No ajuste por mínimos quadrados escolhemos uma família simples de funções e procuramos a que deixa mínimo o resíduo (slide 2)

$$R = \sum_{j=1}^{N}\Big(f(x_j) - y_j\Big)^2.$$

Para um polinômio de grau m, $p_m(x) = \sum_{k=0}^{m} a_k x^k$, a soma dos quadrados dos erros nos n pontos é (slide 10)

$$E(a_0, \ldots, a_m) = \sum_{i=1}^{n}\Big[p_m(x_i) - y_i\Big]^2 = \sum_{i=1}^{n}\Big[\sum_{k=0}^{m} a_k x_i^k - y_i\Big]^2.$$

No mínimo, todas as derivadas parciais se anulam (slide 11):

$$\frac{\partial E}{\partial a_j} = \sum_{i=1}^{n} 2\Big[\sum_{k=0}^{m} a_k x_i^k - y_i\Big]x_i^j = 0, \qquad j = 0, 1, \ldots, m.$$

e, isolando as incógnitas, chegamos às equações normais (slide 12):

$$\begin{array}{rcrcccrcl} a_0\, n & + & a_1 \sum x_i & + & \cdots & + & a_m \sum x_i^m & = & \sum y_i \\ a_0 \sum x_i & + & a_1 \sum x_i^2 & + & \cdots & + & a_m \sum x_i^{m+1} & = & \sum y_i x_i \\ \vdots & & \vdots & & & & \vdots & & \vdots \\ a_0 \sum x_i^m & + & a_1 \sum x_i^{m+1} & + & \cdots & + & a_m \sum x_i^{2m} & = & \sum y_i x_i^m \end{array}$$

Na forma matricial, isso é um sistema $Aa = b$ de tamanho $(m+1) \times (m+1)$, com

$$A_{jk} = \sum_{i=1}^{n} x_i^{j+k}, \qquad b_j = \sum_{i=1}^{n} x_i^j\, y_i, \qquad j, k = 0, \ldots, m,$$

que é a mesma montagem do laboratório de 13/10 (SALLES NETO, 2026c). Como no livro, chamamos o valor mínimo de E de *erro* do ajuste. Quanto menor o E, mais perto dos pontos a curva passa.

Algumas propriedades de E ajudam a ler os resultados da parte 2. O mínimo de E nunca aumenta quando o grau sobe, porque um polinômio de grau m também é um polinômio de grau $m+1$ com $a_{m+1} = 0$. Se os pontos estão exatamente sobre um polinômio de grau $\leq m$, ele dá $E = 0$, que é o menor valor possível, e o ajuste devolve esse polinômio. E quando o número de coeficientes é igual ao número de pontos ($m + 1 = n$), o ajuste vira interpolação, porque o polinômio interpolador passa por todos os pontos e também dá $E = 0$.

3.2 Algoritmo

Para a interpolação na forma de Newton, os passos são:

1. Coloque os dados y_i na coluna 0 da tabela.
2. Para cada ordem $k = 1, \ldots, n$ e cada linha $i = k, \ldots, n$, calcule $\text{tab}[i,k] = \big(\text{tab}[i,k-1] - \text{tab}[i-1,k-1]\big)/(x_i - x_{i-k})$.
3. Pegue os coeficientes na diagonal: $c_k = \text{tab}[k,k]$.
4. Para avaliar $p(x)$, comece com $p = c_n$ e, para $k = n-1, \ldots, 0$, faça $p \leftarrow p\,(x - x_k) + c_k$.

O passo 4 usa a forma encaixada $p(x) = c_0 + (x - x_0)\big[c_1 + (x - x_1)\big[c_2 + \cdots\big]\big]$, e assim cada avaliação custa só n multiplicações, como no esquema de Horner.

Nos mínimos quadrados, montamos a matriz A e o vetor b das equações normais, resolvemos $Aa = b$ por eliminação de Gauss com pivoteamento parcial e retrossubstituição (SALLES NETO, 2026d) e, com os coeficientes, calculamos os resíduos $r_i = y_i - p_m(x_i)$ e o erro $E = \sum r_i^2$.

Os dois métodos são diretos, sem critério de parada nem chute inicial. A interpolação exige nós distintos. Nos mínimos quadrados é preciso ter pelo menos $m+1$ valores distintos de x para que A seja inversível. O interpolador de grau alto pode oscilar e não serve para extrapolar. Já as equações normais ficam mal condicionadas quando os x são grandes ou o grau é alto, porque A herda o problema da matriz de Vandermonde (SALLES NETO, 2026c).

4 Implementação computacional

O código foi escrito em Python 3.11, com `numpy` para vetores e matrizes e `matplotlib` para os gráficos. Ele está na pasta `codigo/` e no repositório indicado no Apêndice A. As funções dos métodos ficam em `metodos.py`, e cada parte do TC tem o seu script, `parte1_interpolacao.py` e `parte2_minimos_quadrados.py`. Os scripts rodam sozinhos, imprimem todos os números usados neste relatório e salvam as figuras em `figuras/`.

Na interpolação, `diferencas_divididas` monta a tabela pela regra do slide 19 e `avalia_newton` calcula $p(x)$ pelo passo 4 do algoritmo (Listagem 1). A função `newton_para_potencias` expande o polinômio em potências de x; só a usamos nos exercícios 1 e 2. Para conferir, no exercício 1 resolvemos também o sistema de Vandermonde, e nos exercícios 3 e 4 comparamos com a função `interpola` que veio pronta no notebook do laboratório (SALLES NETO, 2026c), que calcula o mesmo polinômio pela fórmula baricêntrica de Lagrange.

Para os mínimos quadrados reaproveitamos as funções que o grupo escreveu no laboratório de 13/10 (`equacoes_normais`, `resolve`, `avalia` e `soma_quadrados`), em que a `resolve` faz Gauss com pivoteamento parcial. A única função nova é a `ajusta`, que chama as duas primeiras (Listagem 2). Os coeficientes obtidos foram comparados com os do `numpy.polyfit` (NUMPY DEVELOPERS, 2026).

As contas são feitas em ponto flutuante de precisão dupla (`float64`), com $\epsilon_{\text{máq}} \approx 2{,}2 \times 10^{-16}$.

4.1 Trecho de código

```
def diferencas_divididas(x, y):
    x = np.asarray(x, float)
    n = len(x)
    tabela = np.zeros((n, n))
    tabela[:, 0] = y                        # ordem 0: os dados
    for k in range(1, n):                   # ordem k
        for i in range(k, n):
            # (vizinha à esquerda - a de cima dela) / (x_i - x_(i-k))
            tabela[i, k] = ((tabela[i, k - 1] - tabela[i - 1, k - 1])
                            / (x[i] - x[i - k]))
    return tabela
```

```
12
13
14 def avalia_newton(c, x_nos, x):
15     x = np.asarray(x, float)
16     p = np.full_like(x, c[-1])
17     for k in range(len(c) - 2, -1, -1):
18         p = p * (x - x_nos[k]) + c[k]
19     return p
```

Listagem 1: Diferenças divididas e avaliação da forma de Newton (trecho de `metodos.py`, sem as docstrings).

```
1  def equacoes_normais(t, y, grau):
2      t = np.asarray(t, float)
3      y = np.asarray(y, float)
4      n = grau + 1                # número de coeficientes a0, ..., a_grau
5      A = np.zeros((n, n))
6      b = np.zeros(n)
7      for j in range(n):          # a linha j vem de dE/da_j = 0
8          for k in range(n):
9              A[j, k] = np.sum(t**(j + k))
10         b[j] = np.sum(t**j * y)
11     return A, b
12
13
14 def ajusta(x, y, grau):
15     A, b = equacoes_normais(x, y, grau)
16     return resolve(A, b)        # Gauss com pivoteamento parcial
```

Listagem 2: Montagem das equações normais e ajuste (trecho de `metodos.py`, sem as docstrings).

5 Experimentos numéricos

5.1 Configuração dos testes

Os dados de todos os exercícios estão no Apêndice B, e a Tabela 1 resume o que cada um pede. Na parte 1 o grau máximo do interpolador é o número de pontos menos um. As estimativas foram avaliadas pelo erro $p(x) - (\text{valor conhecido})$ e pelo erro relativo $|p(x) - \text{valor conhecido}|/\text{valor conhecido}$. Na parte 2 usamos o erro E definido na Seção 3. Como os métodos são diretos, não há chute inicial, tolerância ou número máximo de iterações a registrar.

Tabela 1: Resumo dos exercícios.

Parte	Ex.	Pontos	Grau	O que foi pedido
1	1	4	≤ 3	polinômio interpolador
1	2	6	≤ 5	forma de Newton e $f(4{,}5)$ (exato: 71,375)
1	3	8	≤ 7	$P(5)$, $P(45)$, $P(95)$ e comparação
1	4	7	≤ 6	$C(645)$, $C(795)$, $C(845)$ e comparação
2	2	7	2 e 3	ajustes e erro E
2	3	4	2 e 3	ajustes e comparação com $x^2 + 3x + 7$
2	4	5	3	ajuste de $\text{sen}(\pi x)$
2	5	6	2	ajuste e estimativa de $f(6{,}0)$
2	6	6	2	ajuste

5.2 Resultados

5.2.1 Parte 1, exercício 1

A Tabela 2 é a tabela de diferenças divididas no formato do slide 24, com os coeficientes em negrito na diagonal: $c_0 = 0{,}7$, $c_1 = 74/9 \approx 8{,}2222$, $c_2 = 610/81 \approx 7{,}5309$ e $c_3 = -275/243 \approx -1{,}1317$.

Tabela 2: Parte 1, exercício 1: tabela de diferenças divididas.

x_i	y_i	ordem 1	ordem 2	ordem 3
1,2	**0,7**			
2,1	8,1	**8,2222**		
3,0	27,7	21,7778	**7,5309**	
3,6	45,1	29,0000	4,8148	**−1,1317**

Com esses coeficientes o polinômio interpolador fica

$$p_3(x) = 0{,}7 + 8{,}2222\,(x-1{,}2) + 7{,}5309\,(x-1{,}2)(x-2{,}1) - 1{,}1317\,(x-1{,}2)(x-2{,}1)(x-3{,}0),$$

e, expandindo,

$$\boxed{p_3(x) = -1{,}1317\,x^3 + 14{,}6605\,x^2 - 30{,}6852\,x + 18{,}3667.}$$

Os valores de p_3 nos quatro nós batem com os y_i. O sistema de Vandermonde (slide 9) deu os mesmos coeficientes, com diferença máxima de $1{,}2 \times 10^{-13}$.

5.2.2 Parte 1, exercício 2

Na Tabela 3, todas as diferenças de ordem 3 valem 1 e as de ordem 4 e 5 são zero. Os coeficientes são $c_0 = 14{,}5$, $c_1 = 5$, $c_2 = 3$, $c_3 = 1$ e $c_4 = c_5 = 0$.

Tabela 3: Parte 1, exercício 2: tabela de diferenças divididas.

x_i	$f(x_i)$	ordem 1	ordem 2	ordem 3	ordem 4	ordem 5
1,0	**14,5**					
2,0	19,5	**5**				
3,0	30,5	11	**3**			
4,0	53,5	23	6	**1**		
5,0	94,5	41	9	1	**0**	
6,0	159,5	65	12	1	0	**0**

Mesmo podendo ter grau até 5, o polinômio ficou com grau 3:

$$p(x) = 14{,}5 + 5\,(x-1) + 3\,(x-1)(x-2) + (x-1)(x-2)(x-3) = x^3 - 3x^2 + 7x + 9{,}5.$$

Em $x = 4{,}5$,

$$\boxed{p(4{,}5) = 14{,}5 + 5\,(3{,}5) + 3\,(3{,}5)(2{,}5) + (3{,}5)(2{,}5)(1{,}5) = 71{,}375,}$$

que coincide com o valor exato $f(4{,}5) = 71{,}375$. O programa calculou erro zero.

5.2.3 Parte 1, exercício 3

Com 8 pontos o interpolador tem grau ≤ 7:

$$p_7(T) = c_0 + c_1 T + c_2\,T(T-10) + \cdots + c_7\,T(T-10)(T-20)(T-30)(T-40)(T-60)(T-80).$$

Deixamos o polinômio na forma de Newton, que é como o método o entrega e como o avaliamos. Os coeficientes estão na Tabela 4 e as estimativas na Tabela 5. A diferença para a função `interpola` do laboratório foi de $4{,}4 \times 10^{-16}$.

Tabela 4: Parte 1, exercício 3: coeficientes da forma de Newton.

k	T_k	c_k	k	T_k	c_k
0	0	$6{,}1000 \times 10^{-3}$	4	40	$6{,}2500 \times 10^{-9}$
1	10	$6{,}2000 \times 10^{-4}$	5	60	$-1{,}3889 \times 10^{-12}$
2	20	$2{,}4500 \times 10^{-5}$	6	80	$6{,}4236 \times 10^{-13}$
3	30	$5{,}0000 \times 10^{-7}$	7	100	$-9{,}8380 \times 10^{-15}$

Tabela 5: Parte 1, exercício 3: estimativas da pressão de vapor (em bar).

T (°C)	$p_7(T)$	valor conhecido	erro	erro relativo
5	0,008691	0,008721	−0,000030	0,34%
45	0,095919	0,095848	+0,000071	0,07%
95	0,846173	0,845280	+0,000893	0,11%

5.2.4 Parte 1, exercício 4

Com 7 pontos o polinômio tem grau ≤ 6. Os coeficientes estão na Tabela 6 e as estimativas na Tabela 7. Como a diferença para a `interpola` foi de $1{,}1 \times 10^{-16}$, o erro de 14% em $C(645)$ não é erro de conta. Ele vem do próprio polinômio interpolador, como mostramos na Seção 6.

Tabela 6: Parte 1, exercício 4: coeficientes da forma de Newton.

k	T_k	c_k	k	T_k	c_k
0	605	$6{,}2200 \times 10^{-1}$	4	825	$2{,}0402 \times 10^{-10}$
1	685	$4{,}1250 \times 10^{-4}$	5	855	$7{,}2276 \times 10^{-12}$
2	725	$-7{,}2917 \times 10^{-7}$	6	875	$1{,}8482 \times 10^{-13}$
3	765	$6{,}5104 \times 10^{-10}$			

Tabela 7: Parte 1, exercício 4: estimativas da propriedade C do titânio.

T	$p_6(T)$	valor conhecido	erro	erro relativo
645	0,549294	0,639	−0,089706	14,04%
795	0,700127	0,694	+0,006127	0,88%
845	0,810872	0,812	−0,001128	0,14%

5.2.5 Parte 2, exercício 2

Como exemplo de montagem, as equações normais do item (a), com $n = 7$ pontos e $m = 2$, são as seguintes (os outros sistemas estão no Apêndice C):

$$\begin{bmatrix} 7 & 12{,}8 & 25{,}94 \\ 12{,}8 & 25{,}94 & 56{,}972 \\ 25{,}94 & 56{,}972 & 132{,}725 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 293{,}1 \\ 688{,}69 \\ 1681{,}831 \end{bmatrix}.$$

Os dois ajustes deram

$$\text{(a)}\quad p_2(x) = 29{,}9184\,x^2 - 52{,}3429\,x + 26{,}7151, \qquad E = 67{,}9734;$$
$$\text{(b)}\quad p_3(x) = -15{,}5148\,x^3 + 118{,}6371\,x^2 - 211{,}5061\,x + 115{,}2626, \qquad E = 31{,}7280.$$

Os coeficientes com seis casas estão na Tabela 8 e os resíduos $r_i = y_i - p(x_i)$ de cada ajuste, na Tabela 9.

Tabela 8: Parte 2, exercício 2: coeficientes e erro E.

	(a) grau 2	(b) grau 3
a_0	26,715053	115,262642
a_1	−52,342931	−211,506090
a_2	29,918433	118,637147
a_3	—	−15,514782
E	67,973396	31,728025

Tabela 9: Parte 2, exercício 2: valores ajustados e resíduos.

x_i	y_i	$p_2(x_i)$	r_i (grau 2)	$p_3(x_i)$	r_i (grau 3)
1,0	5,5	4,2906	+1,2094	6,8789	−1,3789
1,2	7,3	6,9861	+0,3139	5,4833	+1,8167
1,5	14,1	15,5171	−1,4171	12,5747	+1,5253
1,8	26,3	29,4335	−3,1335	28,4538	−2,1538
2,1	48,2	48,7352	−0,5352	50,6073	−2,4073
2,4	80,1	73,4222	+6,6778	76,5217	+3,5783
2,8	111,6	114,7154	−3,1154	112,5803	−0,9803

5.2.6 Parte 2, exercício 3

Os pontos foram gerados por $f(x) = x^2 + 3x + 7$. No grau 2 o ajuste deu $a_0 = 7$, $a_1 = 3$ e $a_2 = 1$, que é o próprio f, com $E = 5{,}7 \times 10^{-29}$. No grau 3 vieram os mesmos três coeficientes, mais $a_3 = 2{,}9 \times 10^{-14}$, e $E = 1{,}4 \times 10^{-27}$. Esses dois valores de E são zero a menos de arredondamento. A Tabela 10 compara os dois polinômios com os valores exatos.

Tabela 10: Parte 2, exercício 3: polinômios de mínimos quadrados e valores exatos.

x	$f(x)$ exato	$p_2(x)$	$p_3(x)$
0	7	7,0000000000	7,0000000000
1	11	11,0000000000	11,0000000000
2	17	17,0000000000	17,0000000000
3	25	25,0000000000	25,0000000000

A maior diferença para os valores exatos foi de $7{,}1 \times 10^{-15}$ no grau 2 e de $2{,}8 \times 10^{-14}$ no grau 3.

5.2.7 Parte 2, exercício 4

Nos pontos $x = -1;\ -0{,}5;\ 0;\ 0{,}5;\ 1$ os valores são $y = \operatorname{sen}(\pi x) = 0;\ -1;\ 0;\ 1;\ 0$. As equações normais de grau 3 ficam

$$\begin{bmatrix} 5 & 0 & 2{,}5 & 0 \\ 0 & 2{,}5 & 0 & 2{,}125 \\ 2{,}5 & 0 & 2{,}125 & 0 \\ 0 & 2{,}125 & 0 & 2{,}03125 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0{,}25 \end{bmatrix}.$$

Os zeros aparecem porque os pontos são simétricos em relação a $x = 0$, então as somas de potências ímpares de x se anulam. O sistema se separa em dois. As linhas 1 e 3 só têm a_0 e a_2 e o lado direito é zero, então $a_0 = a_2 = 0$. As linhas 2 e 4 dão

$$\begin{bmatrix} 2{,}5 & 2{,}125 \\ 2{,}125 & 2{,}03125 \end{bmatrix} \begin{bmatrix} a_1 \\ a_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0{,}25 \end{bmatrix} \quad \Longrightarrow \quad a_1 = \frac{8}{3}, \qquad a_3 = -\frac{8}{3}.$$

O programa deu $a_1 = 2{,}666667$ e $a_3 = -2{,}666667$. Os coeficientes a_0 e a_2 ficaram da ordem de 10^{-16} em vez de zero porque, no computador, $\operatorname{sen}(\pm\pi)$ não dá exatamente zero. O polinômio é

$$\boxed{p_3(x) = \frac{8}{3}\,x - \frac{8}{3}\,x^3 \approx 2{,}6667\,x - 2{,}6667\,x^3,}$$

e o erro $E = 4{,}3 \times 10^{-31}$ mostra que ele passa pelos 5 pontos.

5.2.8 Parte 2, exercício 5

O polinômio de grau 2 ajustado é

$$p_2(x) = 1{,}9465\,x^2 - 9{,}9928\,x + 12{,}5889, \qquad E = 22{,}1262,$$

e a estimativa pedida é

$$\boxed{f(6{,}0) \approx p_2(6{,}0) = 22{,}7046.}$$

Com seis casas, os coeficientes são $a_0 = 12{,}588932$, $a_1 = -9{,}992767$ e $a_2 = 1{,}946451$. Os resíduos estão na Tabela 11.

5.2.9 Parte 2, exercício 6

Para os dados do exercício 6, a parábola de mínimos quadrados é

$$\boxed{p_2(x) = 0{,}4177\,x^2 + 1{,}6250\,x - 3{,}1162,} \qquad E = 26{,}4826,$$

com $a_0 = -3{,}116215$, $a_1 = 1{,}625013$ e $a_2 = 0{,}417703$. Os resíduos estão na mesma Tabela 11.

Tabela 11: Parte 2, exercícios 5 e 6: valores ajustados e resíduos.

Exercício 5				Exercício 6			
x_i	y_i	$p_2(x_i)$	r_i	x_i	y_i	$p_2(x_i)$	r_i
1,2	2,3	3,4005	−1,1005	1,0	0,1	−1,0735	+1,1735
3,0	3,3	0,1287	+3,1713	2,3	1,5	2,8310	−1,3310
4,5	5,6	7,0371	−1,4371	3,8	7,4	9,0905	−1,6905
5,6	15,4	17,6701	−2,2701	4,9	15,2	14,8754	+0,3246
6,9	38,2	36,3094	+1,8906	6,1	26,2	22,3391	+3,8609
8,1	59,1	59,3542	−0,2542	7,2	27,9	30,2376	−2,3376

Todos os coeficientes da parte 2 bateram com os do `numpy.polyfit`. A maior diferença, $1{,}5 \times 10^{-9}$, apareceu no grau 3 do exercício 2; nos demais casos ficou abaixo de 10^{-11}.

5.3 Gráficos e figuras

Nas Figuras 1 e 2 estão os polinômios da parte 1 e os ajustes da parte 2, desenhados junto com os dados.

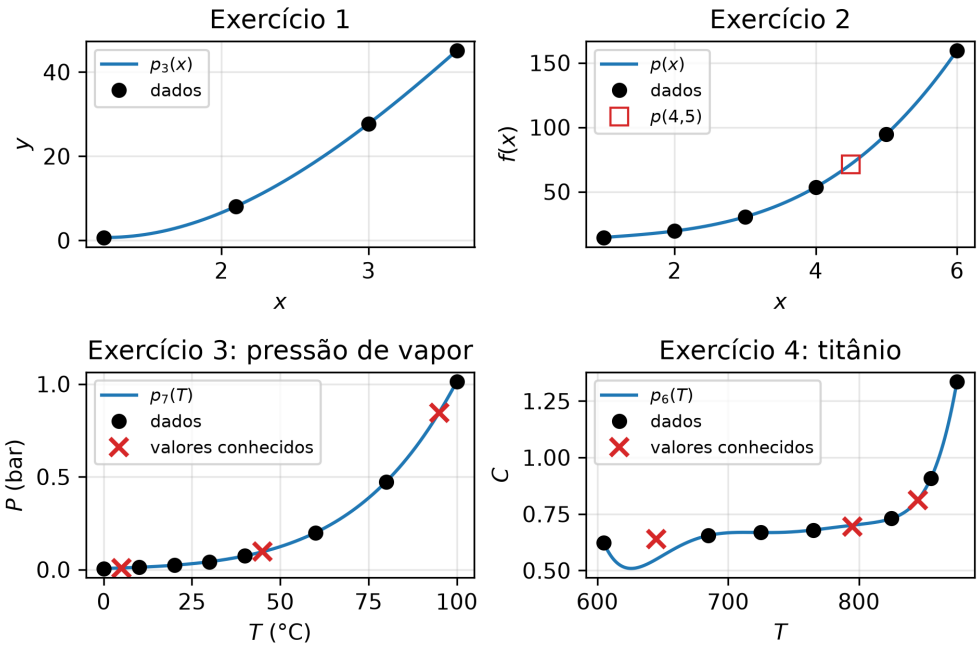


Figura 1: Parte 1: dados e polinômios interpoladores. Nos exercícios 3 e 4, os "x" vermelhos são os valores conhecidos dados no enunciado.

Fonte: elaboração própria.

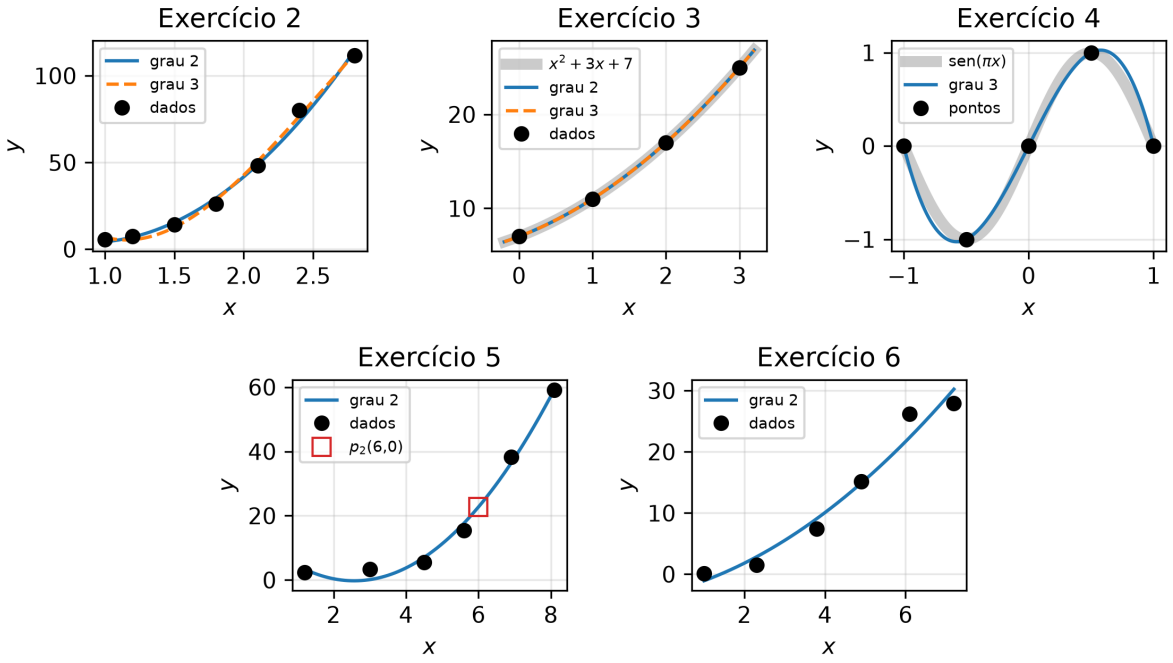


Figura 2: Parte 2: dados e polinômios de mínimos quadrados. No exercício 3 as curvas de grau 2 e 3 ficam em cima de $f(x) = x^2 + 3x + 7$ (faixa cinza); no exercício 4 a faixa cinza é $\text{sen}(\pi x)$.

Fonte: elaboração própria.

6 Análise e discussão

6.1 Parte 1: interpolação

No exercício 1 não houve surpresa. Com 4 pontos e grau 3, a tabela de Newton e o sistema de Vandermonde deram o mesmo polinômio, como tinha que acontecer pela unicidade (slide 8).

O resultado exato do exercício 2 também tem explicação. Como as diferenças de ordem 3 são todas iguais a 1 e as de ordem 4 e 5 são nulas, os dados vêm de um polinômio de grau 3, $f(x) = x^3 - 3x^2 + 7x + 9{,}5$, e pela unicidade o interpolador é o próprio f. A fórmula do erro do slide 28 leva à mesma conclusão, porque com 6 nós ela depende de $f^{(6)}$, que é zero para um polinômio de grau 3. Na prática, olhando as colunas da tabela de Newton já dá para perceber quando os dados vêm de um polinômio de grau baixo.

No exercício 3 o maior erro relativo foi de 0,34%. Em $T = 5$ e $T = 45$ os erros absolutos (0,000030 e 0,000071 bar) são da ordem da última casa decimal da tabela. A pressão foi dada com 4 casas, então cada valor de entrada já pode ter até 0,00005 bar de arredondamento, e nesses dois pontos não faz sentido esperar uma estimativa mais precisa que os próprios dados. O maior erro absoluto apareceu em $T = 95$, o ponto mais perto da ponta da tabela, o que combina com a cota do slide 29. O produto $|\prod(T - T_i)|$ vale $5{,}7 \times 10^{12}$ em $T = 95$, cerca de 44 vezes o valor em $T = 5$ ($1{,}3 \times 10^{11}$) e 67 vezes o valor em $T = 45$ ($8{,}5 \times 10^{10}$). Ele é maior em $T = 95$ porque ali os nós estão espaçados de 20 °C (60, 80 e 100), contra 10 °C no começo da tabela. Já o maior erro relativo ficou em $T = 5$ porque a pressão ali é pequena (cerca de 0,009 bar), e mesmo um erro absoluto pequeno vira uma porcentagem maior.

O número de condição da matriz de Vandermonde deste exercício deu $1{,}3 \times 10^{14}$. Resolvendo o sistema do slide 9, poderíamos perder uns 14 dos cerca de 16 algarismos do `float64`. A tabela de Newton não usa esse sistema.

O exercício 4 foi o único com erro grande. Em $T = 645$ o polinômio deu 0,5493, contra 0,639 do valor conhecido, um erro de 14%. A Figura 1 mostra por quê: os dados são crescentes, mas o polinômio de grau 6 desce logo depois de $T = 605$ e chega a 0,509 em $T \approx 626$, abaixo do primeiro ponto. É a mesma oscilação do fenômeno de Runge (slides 32 e 33), e aqui ela tem duas causas. A primeira é o salto no fim da tabela, de 0,907 para 1,336 entre $T = 855$ e $T = 875$. Para acompanhar esse salto, um único polinômio de grau 6 precisa entortar muito, ou seja, M_7 é grande. A segunda é a posição do ponto. O valor 645 fica no meio do maior intervalo entre nós (de 605 a 685, 80 unidades), enquanto no fim da tabela os nós estão a 20 ou 30 unidades um do outro. Em $T = 645$ o produto $|\prod(T - T_i)|$ é $1{,}3 \times 10^{14}$, cerca de 21 vezes o de $T = 795$ e 60 vezes o de $T = 845$, e nesses dois pontos as estimativas ficaram boas (erros de 0,88% e 0,14%).

Testamos também interpolar só com os 4 nós mais próximos de cada ponto, o que dá um polinômio de grau 3. Para $T = 645$, com os nós 605, 685, 725 e 765, a estimativa foi

0,63975, com erro de 0,12%. Em $T = 795$ e $T = 845$, porém, o grau 3 errou mais que o grau 6 (2,75% e 1,51%), então nenhuma das duas opções ganha nos três pontos. Com nós mal distribuídos e um salto nos dados, é preciso olhar o gráfico do polinômio antes de usar a estimativa. Os *splines*, citados no slide 4, usam um polinômio de grau baixo em cada trecho da tabela e seriam uma alternativa, mas ficaram fora deste trabalho.

Neste exercício o número de condição da matriz de Vandermonde passa de 10^{22}, acima de $1/\epsilon_{\text{máq}} \approx 4{,}5 \times 10^{15}$. Pelo sistema do slide 9 não sobraria nenhum algarismo confiável, que é o problema apontado no slide 38 ("Vandermonde é mal condicionado"). A forma de Newton não usa esse sistema, e a conferência com a `interpola` bateu até $1{,}1 \times 10^{-16}$.

6.2 Parte 2: mínimos quadrados

No exercício 2, passar do grau 2 para o grau 3 reduziu o erro de $E = 67{,}97$ para $E = 31{,}73$, um valor 2,1 vezes menor. Que E não aumentaria já era esperado, porque um polinômio de grau 2 é um polinômio de grau 3 com $a_3 = 0$. Na Tabela 9, o maior resíduo do grau 2 é +6,68, em $x = 2{,}4$; no grau 3 o maior resíduo continua nesse ponto, mas cai para +3,58. Um E menor, porém, não garante previsão melhor fora dos pontos. No laboratório de 13/10 a cúbica teve soma dos quadrados menor que a parábola nos dados de treino e mesmo assim errou mais na validação (SALLES NETO, 2026c). Como aqui não há dados extras para testar, a conclusão possível é que o grau 3 passa mais perto dos 7 pontos.

No exercício 3 os dois ajustes devolveram $f(x) = x^2 + 3x + 7$, e os valores nos quatro pontos coincidiram com os exatos, como previsto na Seção 3. No grau 2 os pontos estão exatamente sobre uma parábola, que dá $E = 0$, e não existe ajuste melhor que esse. No grau 3 são 4 coeficientes para 4 pontos, o ajuste vira interpolação e o único polinômio de grau ≤ 3 que passa pelos pontos é o próprio f (slide 8). Por isso a_3 ficou praticamente nulo. As diferenças de 10^{-14} e 10^{-15} são arredondamento.

O resultado do exercício 4 foi o que mais nos surpreendeu, porque um polinômio de grau 3, com 4 coeficientes, passou exatamente por 5 pontos. A causa é a simetria. Com $a_0 = a_2 = 0$ o polinômio é ímpar, como o seno, então acertar $x = 0{,}5$ e $x = 1$ garante acertar $x = -0{,}5$ e $x = -1$, e em $x = 0$ os dois valem zero. Restam duas condições para duas incógnitas,

$$p(1) = a_1 + a_3 = 0, \qquad p(0{,}5) = \frac{a_1}{2} + \frac{a_3}{8} = 1,$$

que dão $a_1 = 8/3$ e $a_3 = -8/3$, o mesmo resultado das equações normais. Passar pelos pontos não significa que p_3 seja igual a $\operatorname{sen}(\pi x)$. Entre os pontos a diferença chega a 0,18, em $x \approx \pm 0{,}81$ (Figura 2), que é o "aproximar $\neq$ interpolar" do slide 7. O polinômio de Taylor de grau 3 em torno de zero, $\pi x - \pi^3 x^3/6$, erra até 2,03 em $[-1, 1]$, já que só é bom perto de $x = 0$. O ajuste distribui o erro pelo intervalo inteiro.

A estimativa do exercício 5, $f(6{,}0) \approx 22{,}70$, fica entre os dados vizinhos (15,4 em $x = 5{,}6$ e 38,2 em $x = 6{,}9$), e nessa região a curva passa perto dos pontos, com resíduos

de $-2{,}27$ e $+1{,}89$. Por estar dentro do intervalo dos dados, consideramos a estimativa razoável; fora dele valeria o cuidado do slide 37. Estranhamos que o vértice da parábola fique em $x \approx 2{,}57$ com valor $-0{,}24$, abaixo de zero, sendo que todos os dados são positivos. Isso explica o maior resíduo ($+3{,}17$) em $x = 3{,}0$, porque uma parábola não consegue ser quase plana no começo e subir rápido no fim ao mesmo tempo.

No exercício 6 o ajuste deu $E = 26{,}48$. Os dados crescem cada vez mais rápido até $x = 6{,}1$ e depois quase param (de 26,2 para 27,9). Como $a_2 > 0$, a parábola tem concavidade para cima em todo o intervalo e não acompanha essa parada. Os maiores resíduos ficam no fim, $+3{,}86$ em $x = 6{,}1$, com a parábola abaixo do dado, e $-2{,}34$ em $x = 7{,}2$, com a parábola acima. Usada para prever valores além de $x = 7{,}2$, essa parábola continuaria subindo cada vez mais rápido, o contrário do que os dois últimos pontos sugerem.

Nos ajustes da parte 2 o número de condição da matriz A das equações normais ficou entre 50 (exercício 4) e $1{,}1 \times 10^6$ (exercício 2, grau 3), bem abaixo de $1/\epsilon_{\text{máq}}$. Isso explica a concordância com o `numpy.polyfit`, que resolve o problema por outro caminho: a maior diferença foi de $1{,}5 \times 10^{-9}$. No laboratório vimos que a situação muda quando os x são grandes, como os anos da série de CO_2, ou quando o grau é alto, porque $A = X^\top X$ herda o mal condicionamento da matriz de Vandermonde (SALLES NETO, 2026c). Aqui os x são pequenos e o grau vai no máximo a 3.

7 Conclusões

A interpolação na forma de Newton funcionou bem quando os nós cobriam o intervalo de forma razoável. No exercício 2 a tabela de diferenças divididas mostrou que os dados vinham de um polinômio de grau 3, e $f(4{,}5) = 71{,}375$ saiu exato. No exercício 3 a pressão de vapor foi estimada com erro abaixo de 0,4%. O exercício 4 foi diferente. Com 80 unidades entre os dois primeiros nós e um salto no fim da tabela, o polinômio de grau 6 oscilou e errou $C(645)$ em 14%, enquanto em $T = 795$ e $T = 845$ o erro ficou abaixo de 1%. Nos exercícios 3 e 4 a matriz de Vandermonde teve número de condição de $1{,}3 \times 10^{14}$ e acima de 10^{22}, o que justificou o uso da forma de Newton.

Nos mínimos quadrados, as equações normais deram os mesmos coeficientes do `numpy.polyfit`, com diferença máxima de $1{,}5 \times 10^{-9}$. Subir do grau 2 para o grau 3 reduziu E de 67,97 para 31,73 no exercício 2, mas sem dados de validação não dá para afirmar que o grau 3 prevê melhor. O exercício 4 da parte 2 mostrou que ter $E = 0$ nos pontos não garante proximidade com a função, já que o ajuste passou pelos 5 pontos e ainda errou 0,18 entre eles. Por isso, em todos os exercícios, conferimos o gráfico além dos números.

A principal limitação é que, nos exercícios 3 a 6, a função que gerou os dados é desconhecida, e a cota de erro do slide 29, que precisa de M_{n+1}, não pode ser calculada.

Para o exercício 4, o próximo passo seria testar interpolação por partes (*splines*). Nos ajustes, faltou comparar graus diferentes com algum critério de validação, como fizemos no laboratório.

8 Referências

KHARAB, A.; GUENTHER, R. B. *An introduction to numerical methods*: a MATLAB approach. 4. ed. Boca Raton: CRC Press, 2018.

NUMPY DEVELOPERS. *numpy.polyfit*. 2026. Documentação do NumPy. Disponível em: <https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html>. Acesso em: 30 out. 2026.

SALLES NETO, L. L. *Ajuste de curvas: mínimos quadrados*. 2026. Slides da aula de 13/10, disciplina Cálculo Numérico, ICT-UNIFESP, São José dos Campos.

SALLES NETO, L. L. *Interpolação polinomial*. 2026. Slides da disciplina Cálculo Numérico, ICT-UNIFESP, São José dos Campos.

SALLES NETO, L. L. *Quando o CO_2 vai passar de 450 ppm? Ajuste de curvas pelo método dos mínimos quadrados*. 2026. Atividade de laboratório da aula de 13/10 e notebook `lab_mmq_colab.ipynb`, disciplina Cálculo Numérico, ICT-UNIFESP, São José dos Campos.

SALLES NETO, L. L. *Resolução de sistemas lineares: métodos diretos e métodos iterativos*. 2026. Slides da disciplina Cálculo Numérico, ICT-UNIFESP, São José dos Campos.

A Código-fonte e instruções de execução

Para rodar é preciso Python 3 com `numpy` e `matplotlib`. Na pasta do trabalho:

```
python codigo/parte1_interpolacao.py
python codigo/parte2_minimos_quadrados.py
```

Os scripts imprimem os resultados no terminal e salvam as figuras em `figuras/`. As saídas usadas neste relatório estão nos arquivos `saida_parte1.txt` e `saida_parte2.txt`. Para compilar o relatório: `pdflatex`, `bibtex`, `pdflatex` e `pdflatex`.

Repositório: github.com/DaviGiuberti/TC3

```
"""Funções do TC3 de Cálculo Numérico: interpolação e mínimos quadrados.

As funções de mínimos quadrados (equacoes_normais, resolve, avalia e
soma_quadrados) são as que o grupo escreveu no laboratório de 13/10.
A função interpola é a que veio pronta no notebook desse laboratório.
"""
import numpy as np

```

```
9
10 # ------------------------------------------------------------ interpolação
11
12 def diferencas_divididas(x, y):
13     """Tabela de diferenças divididas (slides 17 a 24 de interpolação).
14
15     tabela[i, k] = f[x_(i-k), ..., x_i]. A coluna 0 são os dados e os
16     coeficientes da forma de Newton ficam na diagonal.
17     """
18     x = np.asarray(x, float)
19     n = len(x)
20     tabela = np.zeros((n, n))
21     tabela[:, 0] = y                        # ordem 0: os dados
22     for k in range(1, n):                   # ordem k
23         for i in range(k, n):
24             # (vizinha à esquerda - a de cima dela) / (x_i - x_(i-k))
25             tabela[i, k] = ((tabela[i, k - 1] - tabela[i - 1, k - 1])
26                             / (x[i] - x[i - k]))
27     return tabela
28
29
30 def coeficientes_newton(x, y):
31     """c_k = f[x_0, ..., x_k], a diagonal da tabela."""
32     return np.diag(diferencas_divididas(x, y)).copy()
33
34
35 def avalia_newton(c, x_nos, x):
36     """Avalia p(x) = c0 + c1 (x - x0) + ... + cn (x - x0)...(x - x_(n-1)).
37
38     Começa pelo último coeficiente e vai multiplicando por (x - x_k) e
39     somando c_k, como no esquema de Horner.
40     """
41     x = np.asarray(x, float)
42     p = np.full_like(x, c[-1])
43     for k in range(len(c) - 2, -1, -1):
44         p = p * (x - x_nos[k]) + c[k]
45     return p
46
47
48 def newton_para_potencias(c, x_nos):
49     """Coeficientes a0, a1, ..., an de p(x) = a0 + a1 x + ... + an x^n."""
50     P = np.polynomial.polynomial
51     a = np.array([c[-1]])
52     for k in range(len(c) - 2, -1, -1):
53         a = P.polymul(a, [-x_nos[k], 1.0])     # vezes (x - x_k)
54         a = P.polyadd(a, [c[k]])               # mais c_k
55     return a
56
57
58 def interpola(xn, yn, x):
59     """Avalia em x o polinômio que interpola os pontos (xn, yn).
60
61     Função pronta do notebook do laboratório de 13/10. É o mesmo
62     polinômio de Lagrange/Newton, só que calculado pela fórmula
63     baricêntrica.
64     """
65     xn = np.asarray(xn, float)
66     yn = np.asarray(yn, float)
```

```
67      w = np.array([1.0 / np.prod(xn[j] - np.delete(xn, j))
68                    for j in range(len(xn))])
69      x = np.atleast_1d(np.asarray(x, float))
70      p = np.empty_like(x)
71      for k, xk in enumerate(x):
72          dif = xk - xn
73          if np.any(dif == 0):                        # x coincide com um nó
74              p[k] = yn[np.argmin(np.abs(dif))]
75          else:
76              q = w / dif
77              p[k] = (q @ yn) / q.sum()
78      return p
79
80
81  # ------------------------------------------------------ mínimos quadrados
82
83  def equacoes_normais(t, y, grau):
84      """Monta A e b das equações normais para o modelo
85          phi(t) = a0 + a1 t + ... + a_grau t^grau
86      com  A[j, k] = soma_i t_i^(j+k)   e   b[j] = soma_i t_i^j y_i.
87      """
88      t = np.asarray(t, float)
89      y = np.asarray(y, float)
90      n = grau + 1                # número de coeficientes a0, ..., a_grau
91      A = np.zeros((n, n))
92      b = np.zeros(n)
93      for j in range(n):          # a linha j vem de dE/da_j = 0
94          for k in range(n):
95              A[j, k] = np.sum(t**(j + k))
96          b[j] = np.sum(t**j * y)
97      return A, b
98
99
100 def resolve(A, b):
101     """Resolve A a = b por eliminação de Gauss com pivoteamento parcial
102     (aula de sistemas lineares), seguida de retrossubstituição."""
103     A = np.array(A, dtype=float)    # cópias, para não alterar A e b de fora
104     b = np.array(b, dtype=float)
105     n = len(b)
106
107     # eliminação: zera os elementos abaixo da diagonal, coluna por coluna
108     for k in range(n - 1):
109         # pivoteamento parcial: traz para a linha k o maior |A[i, k]|,
110         # com i >= k
111         p = k + np.argmax(np.abs(A[k:, k]))
112         if A[p, k] == 0:
113             raise ValueError("matriz singular")
114         if p != k:
115             A[[k, p]] = A[[p, k]]
116             b[[k, p]] = b[[p, k]]
117         for i in range(k + 1, n):
118             m = A[i, k] / A[k, k]        # multiplicador
119             A[i, k:] = A[i, k:] - m * A[k, k:]
120             b[i] = b[i] - m * b[k]
121     if A[n - 1, n - 1] == 0:
122         raise ValueError("matriz singular")
123
124     # retrossubstituição
```

```
    a = np.zeros(n)
    for i in range(n - 1, -1, -1):
        a[i] = (b[i] - A[i, i + 1:] @ a[i + 1:]) / A[i, i]
    return a


def avalia(coef, t):
    """Avalia phi(t) = coef[0] + coef[1] t + coef[2] t^2 + ..."""
    t = np.asarray(t, float)
    return sum(c * t**k for k, c in enumerate(coef))


def soma_quadrados(coef, t, y):
    """E = soma dos quadrados dos resíduos r_i = y_i - phi(t_i)."""
    r = np.asarray(y, float) - avalia(coef, t)
    return np.sum(r**2)


def ajusta(x, y, grau):
    """Polinômio de mínimos quadrados: monta e resolve as equações
    normais."""
    A, b = equacoes_normais(x, y, grau)
    return resolve(A, b)
```

Listagem 3: Arquivo `codigo/metodos.py`.

```
"""TC3 parte 1: interpolação polinomial (exercícios 1 a 4)."""
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from metodos import (avalia_newton, coeficientes_newton,
                     diferencas_divididas, interpola,
                     newton_para_potencias)

PASTA_FIG = Path(__file__).resolve().parent.parent / "figuras"
PASTA_FIG.mkdir(exist_ok=True)
np.set_printoptions(suppress=True, linewidth=120)


def mostra_tabela(x, tabela):
    """Imprime a tabela de diferenças divididas como no slide 24."""
    n = len(x)
    print("   x_i  " + "".join(f"{'ordem ' + str(k):>13s}"
                               for k in range(n)))
    for i in range(n):
        print(f"{x[i]:6g}  " + "".join(f"{tabela[i, k]:13.6g}"
                                       for k in range(i + 1)))


def mostra_potencias(a):
    termos = [f"{a[k]:+.6g} x^{k}" for k in range(len(a) - 1, -1, -1)]
    print("p(x) =", " ".join(termos))


```

```
33 def cond_vandermonde(x):
34     """Número de condição da matriz de Vandermonde do slide 9."""
35     return np.linalg.cond(np.vander(x, increasing=True))
36
37
38 def compara(nome, x, y, pontos, exatos):
39     """Interpola, estima nos pontos pedidos e compara com os valores
40     conhecidos."""
41     c = coeficientes_newton(x, y)
42     estimado = avalia_newton(c, x, pontos)
43     conferencia = interpola(x, y, pontos)       # função do professor
44     erro_nos = np.max(np.abs(avalia_newton(c, x, x) - y))
45     print(f"\n{nome}: coeficientes c_k da forma de Newton")
46     for k, ck in enumerate(c):
47         print(f"  c{k} = {ck: .6e}")
48     print(f"  maior |p(x_i) - y_i| nos nós: {erro_nos:.1e}")
49     print("  diferença para a função interpola do lab: "
50           f"{np.max(np.abs(estimado - conferencia)):.1e}")
51     print(f"  {'x':>6s} {'p(x)':>10s} {'conhecido':>10s} {'erro':>10s}"
52           f" {'erro rel.':>9s} {'|prod(x-xi)|':>13s}")
53     for xp, pe, ex in zip(pontos, estimado, exatos):
54         prod = np.prod(np.abs(xp - np.asarray(x)))
55         print(f"  {xp:6g} {pe:10.6f} {ex:10.6f} {pe - ex:+10.6f}"
56               f" {abs(pe - ex) / ex:9.2%} {prod:13.3e}")
57     return c, estimado
58
59
60 # ------------------------------------------------------------ Exercício 1
61 print("=" * 70)
62 print("Exercício 1: polinômio de grau <= 3")
63 x1 = np.array([1.2, 2.1, 3.0, 3.6])
64 y1 = np.array([0.7, 8.1, 27.7, 45.1])
65 tab1 = diferencas_divididas(x1, y1)
66 mostra_tabela(x1, tab1)
67 c1 = np.diag(tab1)
68 a1 = newton_para_potencias(c1, x1)
69 mostra_potencias(a1)
70 print("p(x_i) =", avalia_newton(c1, x1, x1), " (dados:", y1, ")")
71
72 # conferência pelo sistema de Vandermonde (slide 9)
73 V1 = np.vander(x1, increasing=True)
74 a1_vander = np.linalg.solve(V1, y1)
75 print("coeficientes pelo sistema de Vandermonde:", a1_vander)
76 print("diferença para a forma de Newton expandida: "
77       f"{np.max(np.abs(a1 - a1_vander)):.1e}")
78
79 # ------------------------------------------------------------ Exercício 2
80 print("\n" + "=" * 70)
81 print("Exercício 2: grau <= 5 por diferenças divididas de Newton")
82 x2 = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0])
83 y2 = np.array([14.5, 19.5, 30.5, 53.5, 94.5, 159.5])
84 tab2 = diferencas_divididas(x2, y2)
85 mostra_tabela(x2, tab2)
86 c2 = np.diag(tab2)
87 print("coeficientes c_k:", c2)
88 a2 = newton_para_potencias(c2, x2)
89 mostra_potencias(a2)
90 p45 = avalia_newton(c2, x2, 4.5)
```

```
print(f"p(4.5) = {p45:.10f}   exato = 71.375   erro = {p45 - 71.375:.1e}")

# ------------------------------------------------------------ Exercício 3
print("\n" + "=" * 70)
print("Exercício 3: pressão de vapor da água")
T3 = np.array([0, 10, 20, 30, 40, 60, 80, 100], float)
P3 = np.array([0.0061, 0.0123, 0.0234, 0.0424,
               0.0738, 0.1992, 0.4736, 1.0133])
pontos3 = np.array([5.0, 45.0, 95.0])
exatos3 = np.array([0.008721, 0.095848, 0.84528])
c3, est3 = compara("Exercício 3 (grau <= 7)", T3, P3, pontos3, exatos3)
print("  número de condição da matriz de Vandermonde: "
      f"{cond_vandermonde(T3):.1e}")

# ------------------------------------------------------------ Exercício 4
print("\n" + "=" * 70)
print("Exercício 4: propriedade C do titânio")
T4 = np.array([605, 685, 725, 765, 825, 855, 875], float)
C4 = np.array([0.622, 0.655, 0.668, 0.679, 0.730, 0.907, 1.336])
pontos4 = np.array([645.0, 795.0, 845.0])
exatos4 = np.array([0.639, 0.694, 0.812])
c4, est4 = compara("Exercício 4 (grau <= 6)", T4, C4, pontos4, exatos4)
print("  número de condição da matriz de Vandermonde: "
      f"{cond_vandermonde(T4):.1e}")
malha4 = np.linspace(605, 875, 2001)
p_malha4 = avalia_newton(c4, T4, malha4)
i_min = np.argmin(p_malha4)
print(f"  menor valor de p em [605, 875]: {p_malha4[i_min]:.4f}"
      f" em T = {malha4[i_min]:.1f}")

# extra: em vez dos 7 pontos, usar só os 4 nós mais próximos (grau 3)
print("\n  Extra: interpolação só com os 4 nós mais próximos (grau <= 3)")
for xp, ex in zip(pontos4, exatos4):
    perto = np.sort(np.argsort(np.abs(T4 - xp))[:4])
    c_loc = coeficientes_newton(T4[perto], C4[perto])
    pe = avalia_newton(c_loc, T4[perto], xp)
    print(f"  T = {xp:g}: nós {T4[perto].astype(int)} -> p = {pe:.6f},"
          f" erro = {pe - ex:+.6f} ({abs(pe - ex) / ex:.2%})")

# ------------------------------------------------------------------ figura
fig, ax = plt.subplots(2, 2, figsize=(6.6, 4.4))

xx = np.linspace(x1[0], x1[-1], 300)
ax[0, 0].plot(xx, avalia_newton(c1, x1, xx), color="C0", label="$p_3(x)$")
ax[0, 0].plot(x1, y1, "o", color="k", label="dados")
ax[0, 0].set_title("Exercício 1")
ax[0, 0].set_xlabel("$x$")
ax[0, 0].set_ylabel("$y$")

xx = np.linspace(x2[0], x2[-1], 300)
ax[0, 1].plot(xx, avalia_newton(c2, x2, xx), color="C0", label="$p(x)$")
ax[0, 1].plot(x2, y2, "o", color="k", label="dados")
ax[0, 1].plot(4.5, p45, "s", mfc="none", ms=8, color="C3",
              label="$p(4{,}5)$")
ax[0, 1].set_title("Exercício 2")
ax[0, 1].set_xlabel("$x$")
ax[0, 1].set_ylabel("$f(x)$")

```

```
xx = np.linspace(T3[0], T3[-1], 500)
ax[1, 0].plot(xx, avalia_newton(c3, T3, xx), color="C0", label="$p_7(T)$")
ax[1, 0].plot(T3, P3, "o", color="k", label="dados")
ax[1, 0].plot(pontos3, exatos3, "x", ms=8, mew=2, color="C3",
              label="valores conhecidos")
ax[1, 0].set_title("Exercício 3: pressão de vapor")
ax[1, 0].set_xlabel("$T$ (°C)")
ax[1, 0].set_ylabel("$P$ (bar)")

ax[1, 1].plot(malha4, p_malha4, color="C0", label="$p_6(T)$")
ax[1, 1].plot(T4, C4, "o", color="k", label="dados")
ax[1, 1].plot(pontos4, exatos4, "x", ms=8, mew=2, color="C3",
              label="valores conhecidos")
ax[1, 1].set_title("Exercício 4: titânio")
ax[1, 1].set_xlabel("$T$")
ax[1, 1].set_ylabel("$C$")

for eixo in ax.flat:
    eixo.grid(alpha=.3)
    eixo.legend(fontsize=8)
plt.tight_layout()
plt.savefig(PASTA_FIG / "parte1_interpolacao.png", dpi=300)
print("\nfigura salva em figuras/parte1_interpolacao.png")
```

Listagem 4: Arquivo `codigo/parte1_interpolacao.py`.

```
"""TC3 parte 2: ajuste por mínimos quadrados (exercícios 2 a 6)."""
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from metodos import ajusta, avalia, equacoes_normais, soma_quadrados

PASTA_FIG = Path(__file__).resolve().parent.parent / "figuras"
PASTA_FIG.mkdir(exist_ok=True)
np.set_printoptions(suppress=True, linewidth=120)


def formata(v):
    """Número com 6 casas, ou em notação científica se for quase zero."""
    return f"{v: .1e}" if v != 0 and abs(v) < 1e-4 else f"{v: .6f}"


def resultado(nome, x, y, grau):
    """Ajusta, imprime A, b, coeficientes, E e confere com np.polyfit."""
    A, b = equacoes_normais(x, y, grau)
    coef = ajusta(x, y, grau)
    E = soma_quadrados(coef, x, y)
    # o polyfit devolve os coeficientes do maior grau para o menor
    conferencia = np.polyfit(x, y, grau)[::-1]
    print(f"\n{nome}")
    print("A =\n", A)
    print("b =", b)
    print(f"  número de condição de A: {np.linalg.cond(A):.1e}")
    for k, a in enumerate(coef):
```

```
        print(f"  a{k} = {formata(a)}")
    print(f"  E = soma dos quadrados dos resíduos = {formata(E)}")
    dif = np.max(np.abs(coef - conferencia))
    print(f"  diferença para np.polyfit: {dif:.1e}")
    return coef, E


def tabela_residuos(x, y, *coefs):
    cab = f"  {'x':>6s} {'y':>8s}"
    for c in coefs:
        cab += f" {'p' + str(len(c) - 1) + '(x)':>9s} {'r':>8s}"
    print(cab)
    for xi, yi in zip(x, y):
        linha = f"  {xi:6g} {yi:8g}"
        for c in coefs:
            p = avalia(c, xi)
            linha += f" {p:9.4f} {yi - p:+8.4f}"
        print(linha)


# ------------------------------------------------------------ Exercício 2
print("=" * 70)
print("Exercício 2")
x2 = np.array([1.0, 1.2, 1.5, 1.8, 2.1, 2.4, 2.8])
y2 = np.array([5.5, 7.3, 14.1, 26.3, 48.2, 80.1, 111.6])
c2a, E2a = resultado("(a) grau 2", x2, y2, 2)
c2b, E2b = resultado("(b) grau 3", x2, y2, 3)
tabela_residuos(x2, y2, c2a, c2b)
print(f"  E caiu de {E2a:.4f} para {E2b:.4f} ({E2a / E2b:.1f} vezes menor)")

# ------------------------------------------------------------ Exercício 3
print("\n" + "=" * 70)
print("Exercício 3: dados gerados por f(x) = x^2 + 3x + 7")
x3 = np.array([0.0, 1.0, 2.0, 3.0])
y3 = x3**2 + 3 * x3 + 7
c3_2, E3_2 = resultado("grau 2", x3, y3, 2)
c3_3, E3_3 = resultado("grau 3", x3, y3, 3)
print(f"  {'x':>3s} {'f(x)':>6s} {'p2(x)':>14s} {'p3(x)':>14s}")
for xi, yi in zip(x3, y3):
    print(f"  {xi:3g} {yi:6g} {avalia(c3_2, xi):14.10f}"
          f" {avalia(c3_3, xi):14.10f}")
for nome, c in [("p2", c3_2), ("p3", c3_3)]:
    dif = np.max(np.abs(avalia(c, x3) - y3))
    print(f"  maior |{nome} - f| nos pontos: {dif:.1e}")

# ------------------------------------------------------------ Exercício 4
print("\n" + "=" * 70)
print("Exercício 4: f(x) = sen(pi x) em 5 pontos, grau 3")
x4 = np.array([-1.0, -0.5, 0.0, 0.5, 1.0])
y4 = np.sin(np.pi * x4)
print("y =", y4)
c4, E4 = resultado("grau 3", x4, y4, 3)
print(f"  8/3 = {8 / 3:.6f}")
print("  p3 nos pontos:", avalia(c4, x4))
malha4 = np.linspace(-1, 1, 2001)
erro4 = np.abs(np.sin(np.pi * malha4) - avalia(c4, malha4))
i_max = np.argmax(erro4)
print(f"  maior |sen(pi x) - p3(x)| em [-1, 1]: {erro4[i_max]:.4f}"
```

```
      f" em x = {malha4[i_max]:+.3f}")
taylor = np.pi * malha4 - np.pi**3 * malha4**3 / 6
print("  maior erro do polinômio de Taylor de grau 3 em [-1, 1]: "
      f"{np.max(np.abs(np.sin(np.pi * malha4) - taylor)):.3f}")

# ------------------------------------------------------------ Exercício 5
print("\n" + "=" * 70)
print("Exercício 5")
x5 = np.array([1.2, 3.0, 4.5, 5.6, 6.9, 8.1])
y5 = np.array([2.3, 3.3, 5.6, 15.4, 38.2, 59.1])
c5, E5 = resultado("grau 2", x5, y5, 2)
tabela_residuos(x5, y5, c5)
print(f"  f(6.0) aproximado por p2(6.0) = {avalia(c5, 6.0):.4f}")
x_v = -c5[1] / (2 * c5[2])                # vértice da parábola
print(f"  vértice da parábola: x = {x_v:.4f}, p2 = {avalia(c5, x_v):.4f}")

# ------------------------------------------------------------ Exercício 6
print("\n" + "=" * 70)
print("Exercício 6")
x6 = np.array([1.0, 2.3, 3.8, 4.9, 6.1, 7.2])
y6 = np.array([0.1, 1.5, 7.4, 15.2, 26.2, 27.9])
c6, E6 = resultado("grau 2", x6, y6, 2)
tabela_residuos(x6, y6, c6)

# ------------------------------------------------------------------ figura
fig = plt.figure(figsize=(7.6, 4.4))
grade = fig.add_gridspec(2, 6)
ax = [fig.add_subplot(grade[0, 0:2]), fig.add_subplot(grade[0, 2:4]),
      fig.add_subplot(grade[0, 4:6]), fig.add_subplot(grade[1, 1:3]),
      fig.add_subplot(grade[1, 3:5])]

xx = np.linspace(x2[0], x2[-1], 300)
ax[0].plot(xx, avalia(c2a, xx), color="C0", label="grau 2")
ax[0].plot(xx, avalia(c2b, xx), "--", color="C1", label="grau 3")
ax[0].plot(x2, y2, "o", color="k", label="dados")
ax[0].set_title("Exercício 2")

xx = np.linspace(-0.2, 3.2, 300)
ax[1].plot(xx, xx**2 + 3 * xx + 7, color="0.6", lw=5, alpha=.5,
           label="$x^2+3x+7$")
ax[1].plot(xx, avalia(c3_2, xx), color="C0", label="grau 2")
ax[1].plot(xx, avalia(c3_3, xx), "--", color="C1", label="grau 3")
ax[1].plot(x3, y3, "o", color="k", label="dados")
ax[1].set_title("Exercício 3")

ax[2].plot(malha4, np.sin(np.pi * malha4), color="0.6", lw=5, alpha=.5,
           label=r"sen$(\pi x)$")
ax[2].plot(malha4, avalia(c4, malha4), color="C0", label="grau 3")
ax[2].plot(x4, y4, "o", color="k", label="pontos")
ax[2].set_title("Exercício 4")

xx = np.linspace(x5[0], x5[-1], 300)
ax[3].plot(xx, avalia(c5, xx), color="C0", label="grau 2")
ax[3].plot(x5, y5, "o", color="k", label="dados")
ax[3].plot(6.0, avalia(c5, 6.0), "s", mfc="none", ms=8, color="C3",
           label="$p_2(6{,}0)$")
ax[3].set_title("Exercício 5")

```

```
xx = np.linspace(x6[0], x6[-1], 300)
ax[4].plot(xx, avalia(c6, xx), color="C0", label="grau 2")
ax[4].plot(x6, y6, "o", color="k", label="dados")
ax[4].set_title("Exercício 6")

for eixo in ax:
    eixo.set_xlabel("$x$")
    eixo.set_ylabel("$y$")
    eixo.grid(alpha=.3)
    eixo.legend(fontsize=7)
plt.tight_layout()
plt.savefig(PASTA_FIG / "parte2_minimos_quadrados.png", dpi=300)
print("\nfigura salva em figuras/parte2_minimos_quadrados.png")
```

Listagem 5: Arquivo `codigo/parte2_minimos_quadrados.py`.

B Dados e problemas de teste

Os dados foram copiados do enunciado do TC3 (exercícios do livro de Kharab e Guenther (2018)). Para reproduzir cada teste basta rodar o script da parte correspondente; os dados já estão escritos dentro dos scripts.

Tabela 12: Dados da parte 1 (interpolação).

Exercício 1								
x	1,2	2,1	3,0	3,6				
y	0,7	8,1	27,7	45,1				
Exercício 2	(valor exato: $f(4{,}5) = 71{,}375$)							
x	1,0	2,0	3,0	4,0	5,0	6,0		
$f(x)$	14,5	19,5	30,5	53,5	94,5	159,5		
Exercício 3	(conhecidos: $P(5) = 0{,}008721$; $P(45) = 0{,}095848$; $P(95) = 0{,}84528$)							
T (°C)	0	10	20	30	40	60	80	100
P (bar)	0,0061	0,0123	0,0234	0,0424	0,0738	0,1992	0,4736	1,0133
Exercício 4	(conhecidos: $C(645) = 0{,}639$; $C(795) = 0{,}694$; $C(845) = 0{,}812$)							
T	605	685	725	765	825	855	875	
C	0,622	0,655	0,668	0,679	0,730	0,907	1,336	

Tabela 13: Dados da parte 2 (mínimos quadrados).

Exercício 2							
x	1,0	1,2	1,5	1,8	2,1	2,4	2,8
y	5,5	7,3	14,1	26,3	48,2	80,1	111,6
Exercício 3 (gerados por $f(x) = x^2 + 3x + 7$)							
x	0	1	2	3			
$f(x)$	7	11	17	25			
Exercício 4 ($f(x) = \operatorname{sen}(\pi x)$)							
x	-1	$-0{,}5$	0	0,5	1		
Exercício 5							
x	1,2	3,0	4,5	5,6	6,9	8,1	
y	2,3	3,3	5,6	15,4	38,2	59,1	
Exercício 6							
x	1,0	2,3	3,8	4,9	6,1	7,2	
$f(x)$	0,1	1,5	7,4	15,2	26,2	27,9	

C Material complementar

Equações normais $Aa = b$ dos ajustes da parte 2 que não aparecem no texto principal (os sistemas do exercício 2(a) e do exercício 4 estão na Seção 5). Os valores foram impressos pelo script `parte2_minimos_quadrados.py`.

Exercício 2(b), grau 3.

$$\begin{bmatrix} 7 & 12{,}8 & 25{,}94 & 56{,}972 \\ 12{,}8 & 25{,}94 & 56{,}972 & 132{,}725 \\ 25{,}94 & 56{,}972 & 132{,}725 & 322{,}54868 \\ 56{,}972 & 132{,}725 & 322{,}54868 & 808{,}148234 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 293{,}1 \\ 688{,}69 \\ 1681{,}831 \\ 4222{,}6093 \end{bmatrix}$$

Exercício 3, graus 2 e 3. O sistema de grau 2 é o canto superior esquerdo 3×3 do sistema de grau 3:

$$\begin{bmatrix} 4 & 6 & 14 & 36 \\ 6 & 14 & 36 & 98 \\ 14 & 36 & 98 & 276 \\ 36 & 98 & 276 & 794 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 60 \\ 120 \\ 304 \\ 822 \end{bmatrix}$$

Exercício 5, grau 2.

$$\begin{bmatrix} 6 & 29{,}3 & 175{,}27 \\ 29{,}3 & 175{,}27 & 1155{,}419 \\ 175{,}27 & 1155{,}419 & 8047{,}9699 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 123{,}9 \\ 866{,}39 \\ 6325{,}609 \end{bmatrix}$$

Exercício 6, grau 2.

$$\begin{bmatrix} 6 & 25{,}3 & 133{,}79 \\ 25{,}3 & 133{,}79 & 785{,}917 \\ 133{,}79 & 785{,}917 & 4885{,}9475 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 78{,}3 \\ 466{,}85 \\ 2901{,}081 \end{bmatrix}$$